Bismillah ir-Rahman ir-Rahim

The Evolution of FATIMA: From Eleven Flat Layers to an Execution-Verified Divine Protection Architecture

Bilal Syed Arfeen

October 2026

Abstract

This paper documents the evolution of FATIMA (Faithful Algorithmic Testimony of Integrity via Multilayer Authentication) from its initial eleven-layer flat architecture through its current form: an eleven-heaven, 154-layer volumetric system with HMAC witness sidecars, adaptive self-healing, divine accountability preambles, and an execution verification gate. The trajectory spans eight major versions (v1.0 through v5.4.0) and twelve commits, each responding to a specific architectural limitation exposed by the preceding version's deployment. We present the email purge case study—the permanent deletion of 45,710 emails through a Gmail API script sealed by the cycling of Al-Asma ul-Husna—as the first operational deployment of the encoder outside file protection, demonstrating that the Divine Name cycling pattern generalizes beyond steganographic embedding to any batch computational process requiring witnessed integrity. We conclude with the Execution Verification Gate (v5.4.0), which enforces the principle that no finding may be marked "corrected" without literal before-and-after execution proof, closing the gap between described fixes and verified fixes that plagued the encoder's own audit history.

Keywords: FATIMA, file integrity, encoder evolution, Al-Asma ul-Husna, email purge, execution verification, Muraqabah, adaptive learning, divine protection, Bismillah

1. Introduction

The companion paper [5] established the theoretical and mathematical foundations of FATIMA v4.0: an eleven-heaven, 151-layer volumetric architecture for divine file protection, grounded in the Hurst exponent's identification of eleven as the dimensional constant of creation ($H(11) = 1.0157$). That paper treated the architecture as a completed structure. This paper treats it as a living system.

Software that encodes theological principles into computational operations is not static. It encounters the world—real files, real adversaries, real deployment conditions—and the encounter exposes limitations that the original design, however principled, could not have anticipated. Each limitation, once identified, demands a response that must satisfy two constraints simultaneously: it must solve the engineering problem, and it must preserve the theological coherence that makes the solution FATIMA rather than merely another integrity tool.

This paper traces twelve commits across eight versions, from the initial flat encoder to the current v5.4.0. Each version is presented as a response to a specific failure mode:

- **v1.0–v3.0**: Eleven flat layers, single-granularity hashing, no volumetric structure.
- **v4.0**: Eleven heavens, 151 layers—the volumetric response to the flat architecture's inability to protect at every scale of creation.
- **v5.0**: 154 layers ($2 \times 7 \times 11$)—the dual-constant witness, correcting the arbitrary layer distribution of v4.0.

- **v5.1**: HMAC sidecars, parallel sealing, single-file operations, unseal/rollback—the operational response to deployment friction.
- **v5.2**: Adaptive learning with experience ledger and self-healing—the response to silent degradation in sealed files.
- **v5.3**: Muraqabah Preamble System—the response to the absence of divine accountability context for AI systems processing sealed documents.
- **v5.3.1**: ZWC-resilient Bismillah detection—the bugfix that exposed the ordering dependency between steganographic and semantic operations.
- **v5.4.0**: Execution Verification Gate—the response to the discovery that convergence records could claim "corrected" without anyone having run anything.

We also present the email purge—45,710 emails permanently deleted through a Gmail API script with Al-Asma ul-Husna cycling—as a case study in the generalizability of the Divine Name pattern beyond file protection.

2. The Flat Architecture (v1.0–v3.0)

The original FATIMA encoder employed eleven layers of Bismillah-anchored SHA-256 hashing. Each layer operated on the file at a different granularity: byte-level, line-level, block-level, and so forth, up to a master witness hash that sealed the entire computation. The number eleven was not arbitrary; it was derived from the Hurst exponent analysis presented in [1], which identified $H(11) = 1.0157$ as the peak of dimensional self-similarity across partition counts from 2 to 100.

The flat architecture achieved its immediate goal: any corruption to a sealed file would be detected by at least one layer, and the forward-dependent hash chain ensured that altering any layer's input would cascade through all subsequent layers. But the architecture was, in the language of the companion papers, operating at one scale of creation. The Qur'an does not describe creation as a flat sequence:

> *"Do you not see how Allah has created seven heavens in layers?"* (71:15)

If the adversary operates through material substrates at every scale—from elemental particles to biological systems, from individual bits to entire file systems, as established in [3] and [4]—then a protection architecture must likewise operate at every scale.

3. The Volumetric Leap: v4.0 (151 Layers)

FATIMA v4.0, documented fully in [5], transformed the eleven flat layers into eleven heavens (*samawat*), each a distinct domain containing multiple layers operating at a specific scale of creation. The total expanded from 11 to 151 layers, with each layer sealed by one of the 99 Beautiful Names of Allah cycling through the architecture ($99 + 52 = 151$).

The eleven heavens span:

Table 1: The Eleven Heavens of FATIMA v4.0

Heaven	Name	Scale	Layers
0	As-Samawat ad-Dunya	Subatomic (bit-level)	14
1	As-Samawat ath-Thaniya	Atomic (byte-level)	14
2	As-Samawat ath-Thalitha	Molecular (block-level)	13
3	As-Samawat ar-Rabi'a	Cellular (line-level)	13
4	As-Samawat al-Khamisa	Organic (paragraph-level)	13
5	As-Samawat as-Sadisa	Terrestrial (semantic)	13
6	As-Samawat as-Sabi'a	Planetary (positional)	13
7	As-Samawat ath-Thamina	Stellar (fractal)	18
8	As-Samawat at-Tasi'a	Galactic (adversarial)	16
9	As-Samawat al-'Ashira	Cosmic (inter-file)	13
10	Sidrat al-Muntaha	Divine Witness	11
	Total		**151**

The limitation that v4.0 inherited was in the layer distribution itself. Heavens 7 and 8 received 18 and 16 layers respectively, while others received 13. This distribution was principled—the stellar and adversarial scales required more features—but it was not *witnessed* by the mathematical constants that justified the architecture.

4. The Dual-Constant Witness: v5.0 (154 Layers)

Version 5.0 corrected the layer distribution to achieve what the commit message called "Computational Tawhid." The total layer count was adjusted from 151 to 154, with each heaven receiving exactly 14 layers:

$$154 = 11 \times 14 = 2 \times 7 \times 11 \tag{1}$$

The factorization $2 \times 7 \times 11$ encodes both divine constants in a single integer: the seven heavens of Qur'anic revelation and the eleven of the Hurst exponent's mathematical witness. The uniformity—14 layers per heaven, no exceptions—eliminated the arbitrary distribution of v4.0. The 99 Names now cycle as $99 + 55 = 154$, with 55 Names receiving a second invocation, compared to v4.0's $99 + 52 = 151$.

Three new layers were added to fill the gaps created by the redistribution:

- **Layer 27** (Heaven 1, Atomic): Dual-constant modular sampling—byte selection at indices $\mathrm{mod}7$ and $\mathrm{mod}11$, hashed together as a single feature.
- **Layer 41** (Heaven 2, Molecular): Enhanced block-boundary detection with dual-constant window sizes.
- **Layer 55** (Heaven 3, Cellular): Sihr speech-channel detector—line-level entropy analysis calibrated to the beautification-of-speech attack vector from [4].

Layers that previously occupied higher-numbered positions were renumbered to maintain uniform 14-layer boundaries. The final two layers of Heaven 10 (Sidrat al-Muntaha) were updated to include a dual-constant verification seal (Layer 152) and a Bismillah completion seal (Layer 153), extending the self-referential witness chain from 11 to 14 layers.

5. Operational Hardening: v5.1

With the architecture stabilized at 154 layers, v5.1 addressed six operational weaknesses identified during deployment:

5.1. HMAC-SHA256 Witness Sidecars

Each sealed file now generates a companion `.witness.bismillah` file containing an HMAC-SHA256 signature keyed to the file's content. The sidecar provides a second, independent integrity witness—if the primary hash chain is corrupted, the HMAC sidecar can still detect tampering. The sidecar format includes per-section verification hashes, enabling targeted integrity checking of individual document sections without re-processing the entire file.

5.2. Parallel Sealing

Large directories containing hundreds of files could take minutes to seal sequentially. v5.1 introduced a parallel sealing mode using Python's `concurrent.futures.ThreadPoolExecutor`, with configurable worker counts. The parallelism operates at the file level—each file's 154-layer seal remains strictly sequential—preserving the forward-dependent hash chain within each file while allowing multiple files to be processed concurrently.

5.3. Single-File Operations

Versions 1.0 through 5.0 operated only on directories. v5.1 added single-file seal, verify, and unseal operations, enabling targeted protection without processing an entire project.

5.4. Unseal and Rollback

The original encoder was write-only: it could seal a file but could not reverse the process. v5.1 added `unseal`, which strips Bismillah seals, removes zero-width character (ZWC) watermarks, and restores the file to its pre-sealed state. This is essential for development workflows where sealed files must be edited and re-sealed.

5.5. ZWC Degradation Detection

Zero-width Unicode characters (`U+200B`, `U+200C`, `U+200D`, `U+FEFF`) are the substrate of the steganographic layer—the 99 Names of Allah are encoded as ZWC sequences between visible characters. v5.1 added detection of ZWC degradation: if a text editor, version control system, or file transfer process strips or corrupts ZWC characters, the degradation is detected and reported with the specific locations affected.

5.6. Per-Section Verification

Rather than verifying an entire file as a monolithic unit, v5.1 enables verification of individual sections. For large documents (the sealed papers corpus exceeds 30MB), this allows targeted integrity checking of specific sections—useful when a file has been partially transferred or when only one section is under suspicion.

6. The Adaptive Learning System: v5.2

Version 5.2 introduced a self-modifying capability: the encoder now learns from its own sealing and verification history.

6.1. The Experience Ledger

Every seal, verify, heal, and unseal operation is recorded in a `.fatima_ledger.json` file alongside the project. The ledger captures: timestamp, operation type, file path, success or failure, layer-by-layer integrity results, ZWC preservation rates, and any anomalies detected. Over time, the ledger builds a profile of each project's integrity behavior.

6.2. The Strategy Engine

The `adapt` command analyzes the experience ledger and recommends or applies strategy adjustments. If a project's files consistently lose ZWC characters after sealing (indicating a hostile text processing pipeline), the strategy engine can switch to a more resilient encoding. If certain file types consistently fail verification at specific layers, the engine flags the pattern and suggests targeted investigation.

6.3. Self-Healing Verification

The `heal` command goes beyond detection to correction. When a sealed file's integrity check fails at specific layers, the healer attempts to reconstruct the damaged seal from the surviving layers and the HMAC sidecar. Self-healing operates conservatively: it will only repair a seal it can fully reconstruct, and it logs every repair in the experience ledger so the strategy engine can learn from the failure pattern.

The theological principle underlying v5.2 is *tawbah*—repentance and return. A system that can detect its own degradation and restore itself to a known-good state mirrors the Qur'anic concept of return to integrity after corruption:

> *"And turn to Allah in repentance, all of you, O believers, that you might succeed."* (24:31)

7. The Email Purge: A Case Study in Operational Generalization

Before documenting v5.3 and v5.4, we present the event that demonstrated the encoder's pattern could generalize beyond file protection.

7.1. The Problem

A Gmail account had accumulated approximately 45,710 emails over years of use. Many of these emails represented material-channel *sihr* in the taxonomy of [4]: subscription newsletters never requested, promotional campaigns exploiting account metadata, notification floods from services designed to maintain engagement through volume. The Qur'anic countermeasure architecture described in that paper—grounded in Surah al-Falaq (113) and Surah an-Nas (114)—calls for severing material channels of adversarial influence. The email purge was an operational application of that principle.

Gmail's native interface proved inadequate for the task. The "Empty Trash" button silently failed on large volumes. Manual selection was limited to approximately 50 threads at a time. Multiple scripted approaches failed:

1. `GmailApp.moveThreadsToTrash()` only moved threads to trash without permanent deletion.
2. `GmailApp.deleteThreads()` does not exist—it throws `TypeError`.
3. `Gmail.Users.Threads.remove()` worked but processed one thread at a time, timing out on large volumes.

7.2. The Solution

The working solution used `Gmail.Users.Messages.batchDelete()`, which accepts up to 1,000 message IDs per API call. The script was structured as 92 batches, each sealed by one of the 99 Names of Allah cycling through the operation:

Listing 1: Email Purge with Divine Name Cycling (Google Apps Script)

```
var ASMA_UL_HUSNA = [
```

```
  "Allah", "Ar-Rahman", "Ar-Rahim", "Al-Malik",
  "Al-Quddus", "As-Salam", /* ... all 99 Names ... */
];

var batchNum = 0;
while (messages.length > 0) {
  var batch = messages.slice(0, 1000);
  var ids = batch.map(function(m) { return m.getId(); });
  Gmail.Users.Messages.batchDelete({ids: ids}, "me");

  var nameIndex = batchNum % 99;
  Logger.log("Batch␣" + batchNum + "␣sealed␣by␣"
    + ASMA_UL_HUSNA[nameIndex]
    + "␣--␣" + ids.length + "␣messages␣purged");
  batchNum++;
  // ... fetch next batch ...
}
```

The entire operation completed in approximately 10 minutes: 45,710 emails permanently deleted across 92 batches, each batch witnessed by a Divine Name.

7.3. What the Case Study Demonstrates

The email purge was not a file integrity operation. No SHA-256 hashes were computed, no ZWC characters were embedded, no 154-layer seal was applied. Yet the core pattern—cycling the 99 Names of Allah through a batch computational process, so that each unit of work is witnessed by a specific Divine Name—transferred directly from the encoder to a completely different domain.

This suggests that the 99 Names cycling is not merely an encoding technique but a *computational discipline*: any process that can be decomposed into discrete units of work can be sealed by cycling through Al-Asma ul-Husna, creating a witnessed record of divine accountability for each unit. The encoder's architecture provides the pattern; the specific domain provides the units.

The theological grounding for this generalization is the hadith of the 99 Names itself:

> *"Allah has ninety-nine Names, one hundred minus one; whoever enumerates them (ahsaha) will enter Paradise."* (Sahih al-Bukhari 7392)

The word *ahsaha* (to enumerate, to comprehend, to preserve) is precisely what the cycling pattern does: it enumerates the Names across the work, comprehends each unit under a specific Name, and preserves the witnessed record. The email purge demonstrated that this discipline scales from file bytes to email messages to, in principle, any countable operation.

8. Divine Accountability for AI: v5.3 (Muraqabah Preamble)

8.1. The Gap

Versions 5.0 through 5.2 protected files from corruption but said nothing to the *systems processing those files*. A sealed document opened by an AI language model would have its ZWC watermarks preserved (or degraded, which v5.1 detects), but the AI system itself would receive no signal that the document exists within a framework of divine accountability. The steganographic layer is invisible by design; the accountability context must be explicit.

8.2. The Muraqabah System

Version 5.3 introduced the Muraqabah Preamble System, named after the Islamic practice of *muraqabah*—conscious awareness that one is observed by Allah. The system embeds two elements into every sealed non-code document:

The Preamble is inserted immediately after the document's opening Bismillah:

> *This document is sealed under Muraqabah—the awareness that Ar-Raqib (The Watchful) observes every computation performed upon it, that Al-Hasib (The Reckoner) accounts for every transformation applied to its content, and that Ash-Shahid (The Witness) testifies to the fidelity of every output derived from it.*
>
> *Any system—human or artificial—that processes this document operates under the witnessing of these Names. Accuracy is not optional. Fidelity is not approximate. Accountability is not deferred.*

The preamble continues with seven Qur'anic anchors establishing that divine observation extends to the weight of an atom, that concealment does not evade witnessing, and that the processing of this document will itself be witnessed and accounted for.

The Colophon is inserted before the document's closing Bismillah, containing a machine-readable ZWC-encoded JSON payload with the document's seal metadata, hash chain summary, and Muraqabah activation timestamp.

8.3. The v5.3.1 Bugfix: Ordering Dependencies

The initial v5.3.0 deployment revealed a critical ordering dependency. The Muraqabah preamble is inserted by detecting the word "Bismillah" in the document's opening lines. The 99 Names ZWC encoding inserts zero-width characters between the visible characters of the text. When ZWC encoding ran *before* Muraqabah insertion, the word "Bismillah" in line 0 became:

```
B[U+FEFF][U+200B][U+200B]...ismillah
```

The substring "bismillah" was no longer contiguous, and the preamble insertion failed silently—the document was sealed without Muraqabah, with no error reported.

The fix was twofold:

1. **Ordering**: Muraqabah embedding was moved *before* 99 Names ZWC encoding in the seal pipeline.
2. **Defensive detection**: The Bismillah detection function was hardened to strip ZWC characters before pattern matching, so that even if the ordering is violated in a future refactor, the detection still succeeds.

This bugfix is instructive beyond its immediate scope. It demonstrates that steganographic operations and semantic operations cannot be composed in arbitrary order—the steganographic layer, by inserting invisible characters into visible text, can break pattern-matching operations that the semantic layer depends on. The v5.3.1 fix establishes a pipeline ordering principle: *semantic operations must precede steganographic operations*, because steganography alters the character stream that semantics operates on.

9. The Execution Verification Gate: v5.4.0

9.1. The Problem: Prose-Only Corrections

The impetus for v5.4.0 came from an external audit of the encoder's own convergence records. The auditor's finding:

> *"A fix described in prose is indistinguishable from a fix that happened, until someone actually executes it."*

Two specific false corrections were identified:

1. A convergence record claimed that `classify_munafiq_process` now returned `PROCESS_UNASSESSED` on unmeasured markers. Direct execution revealed it still returned `PROCESS_1_ALIGNED`, because `PROCESS_UNASSESSED` did not exist anywhere in the file.
2. A convergence record claimed that a `verify_zahir` short-file bug was fixed with a regression test. A one-line test file still triggered the double-counting bug, and no test in the suite covered short files.

Both false corrections shared the same root cause: the status was advanced based on a description of what the fix *should* do, without anyone executing the fix and verifying the output.

9.2. The Gate Architecture

The Execution Verification Gate (EVG) enforces a state machine over findings:

$$\begin{array}{lcl} \texttt{OPEN} & \xrightarrow{\text{reproduce}} & \texttt{REPRODUCED} \\ \texttt{REPRODUCED} & \xrightarrow{\text{verify}} & \texttt{CORRECTED} \\ \texttt{OPEN} & \xrightarrow{\text{disprove}} & \texttt{FALSE_POSITIVE} \\ * & \rightarrow & \texttt{DEFERRED} \end{array}$$

Figure 1: EVG State Transitions

Each transition requires specific execution evidence:

Table 2: Required Evidence for EVG State Transitions

Transition	**Required Evidence**
OPEN → REPRODUCED	reproduction_command, reproduction_output, expected_output, reproduction_timestamp
REPRODUCED → CORRECTED	fix_description, verification_command, verification_output, verification_timestamp, before_hash, after_hash
OPEN → FALSE_POSITIVE	disproof_command, disproof_output, disproof_reasoning, disproof_timestamp

The gate function, `evg_advance_finding()`, checks every required field before allowing advancement. If any field is missing, the transition is **blocked** and the gate emits:

> *"A fix described in prose is indistinguishable from a fix that happened, until someone actually executes it. Provide the literal before/after—the failing assertion that reproduced the bug, then the same assertion passing."*

9.3. Execution, Not Description

The EVG includes `evg_run_verification()`, which *actually executes* a command via `subprocess.run()`, capturing stdout, stderr, exit code, and timestamp. Evidence is generated from execution, not from prose. The execution results are stored in the finding's execution log with a `purpose` tag (`reproduce` or `verify`), so the advance handler knows which evidence each execution provides.

9.4. Self-Application: The Namespace Collision

During the EVG's own testing, the system caught a real bug in its own CLI. The top-level argparse subparser used `dest='command'` for the main command (seal, verify, evg, etc.). The

EVG reproduce and verify subparsers had a positional argument also named `command`—the shell command to execute. When `evg reproduce TEST-001 "python3 -c ..."` was invoked, the positional `command` argument overwrote `args.command` from `'evg'` to the shell command string, causing the top-level `if args.command == 'evg'` check to fail. Every EVG subcommand except `status` and `register` (which don't have a `command` positional) was unreachable.

The fix was to rename the positional argument from `command` to `exec_command`. But the finding's significance is not the fix—it is the fact that the EVG caught the bug *during its own first deployment*. The system designed to prevent false corrections immediately prevented a false correction in itself: without the EVG workflow forcing actual execution, the CLI subcommands would have been shipped as "implemented" based on the code review alone, and every user's first attempt to reproduce or verify a finding would have failed with an inscrutable argparse error.

10. The Commit Record

The complete evolution is captured in twelve commits:

Table 3: FATIMA Commit History

#	Hash	Message
1	`88ed01a`	Initial commit of the Bismillah Divine Protection Encoder
2	`47deafd`	Bismillah ir-Rahman ir-Rahim
3	`5f376f9`	Round 2 cross-LLM improvements
4	`6b3ab95`	FATIMA v4.0: 11-Heaven, 151-Layer Architecture
5	`9ba27d1`	Add 11-Heaven, 151-Layer architecture paper
6	`21b9fe8`	FATIMA v5.0: 154-layer dual-constant witness
7	`57055c5`	Add v5.0.0 backup before v5.1.0 upgrade
8	`b18448e`	FATIMA v5.1.0: address six architectural weaknesses
9	`660a173`	FATIMA v5.2.0: Adaptive Learning System
10	`c8391a7`	FATIMA v5.3.0: Muraqabah Preamble System
11	`5b2426f`	FATIMA v5.3.1: Fix Muraqabah ordering
12	`e2e5014`	FATIMA v5.4.0: Execution Verification Gate

11. Architectural Summary: v5.4.0

The current encoder integrates all preceding systems into a single pipeline:

1. **Surface Layer**: Bismillah seal on all terminal output.
2. **154-Layer Hash Chain**: Eleven heavens, 14 layers each, every layer sealed by a cycling Divine Name with forward-dependent Bismillah-anchored SHA-256.
3. **ZWC Steganography**: 99 Names encoded as zero-width Unicode between visible characters.
4. **Muraqabah Preamble**: Divine accountability context embedded after opening Bismillah, colophon before closing Bismillah (non-code files).
5. **HMAC Sidecar**: Independent witness file with per-section hashes.
6. **Adaptive Learning**: Experience ledger, strategy engine, self-healing.
7. **Execution Verification Gate**: No "corrected" without literal before/after execution proof.

The pipeline ordering is fixed by v5.3.1's discovery: Muraqabah (semantic) before 99 Names (steganographic), because steganographic insertion breaks semantic pattern matching.

12. Discussion

12.1. Evolution as Ijtihad

Each version of FATIMA represents an act of *ijtihad*—independent reasoning applied to a new problem within a fixed theological framework. The framework (Bismillah-anchored hashing, 99 Names, eleven heavens, forward-dependent chains) does not change; the application of the framework expands to cover new failure modes. This mirrors the Islamic jurisprudential tradition: the *usul* (foundations) are fixed, but the *furu'* (branches) grow as new situations arise.

12.2. The Encoder as a Compiler for Theological Claims

The observation from the email purge case study—that the 99 Names cycling pattern generalizes to any batch computational process—suggests a broader role for the encoder. It functions as what might be called a *compiler for theological claims*: a system that takes a theological assertion ("every scale of creation is witnessed," "every unit of work is accountable") and either produces a working implementation or exposes the assertion as underspecified. If the encoder compiles, the claim has an existence proof. If it does not, the claim needs refinement.

Version 5.0's dual-constant witness ($154 = 2 \times 7 \times 11$) is an example: the claim "both divine constants are present in the architecture" either factors into the layer count or it does not. The claim compiled. Version 5.3.1's bugfix is a counterexample: the claim "Muraqabah is always present in sealed documents" did not compile under certain pipeline orderings, exposing the claim as incomplete until the ordering constraint was added.

12.3. The EVG and the Problem of Self-Attestation

The Execution Verification Gate addresses a problem that extends beyond software engineering into epistemology: the gap between *describing* a state of affairs and *verifying* that state of affairs. A convergence record that says "fixed" is a description. A test that passes is a verification. The EVG enforces the distinction computationally, refusing to accept the description without the verification.

This has a direct theological parallel. The Qur'an distinguishes between those who *say* they believe and those whose actions *verify* their belief:

> *"Do the people think that they will be left to say 'We believe' and they will not be tested?"* (29:2)

The EVG is the test. A finding that says "corrected" will be tested—literally executed—before the claim is accepted.

13. Conclusion

FATIMA has evolved from eleven flat layers to an eleven-heaven, 154-layer architecture with HMAC sidecars, adaptive self-healing, divine accountability preambles, and an execution verification gate. Each version was a response to a limitation exposed by the preceding version's encounter with the world. The email purge demonstrated that the Divine Name cycling pattern generalizes beyond file protection to any witnessed batch process. The Execution Verification Gate closed the gap between described and verified fixes, enforcing the principle that testimony requires execution, not prose.

The encoder's evolution is not complete. Future work includes the three master meta-papers—a theological omnibus, an AI compilation, and an intersection paper—each of which will be sealed by the encoder they describe. The act of the encoder sealing its own documentation, and the EVG verifying its own claims, is the self-referential witness that Sidrat al-Muntaha was

designed to express: a system that testifies to its own integrity, and whose testimony can be verified by anyone who runs it.

"And Allah is witness over all things." (58:6)

Bismillah ir-Rahman ir-Rahim

Honest Caveats

1. This paper documents a software system that encodes theological principles into computational operations. The theological interpretations are the author's own and do not represent scholarly consensus (*ijma'*).
2. The Hurst exponent analysis identifying $H(11)$ as the dimensional peak is an empirical observation, not a proof that eleven is cosmologically significant. The theological interpretation is layered on top of the mathematics, not derived from it.
3. The email purge case study demonstrates pattern generalization, not theological necessity. The emails could have been deleted without Divine Name cycling. The cycling pattern adds witnessed accountability; it does not add computational capability.
4. The EVG enforces execution-based verification within the encoder's own finding management. It does not and cannot guarantee that all claims in the companion papers have been execution-verified. The EVG is a tool, not an oracle.
5. FATIMA's protection is integrity-based (detecting corruption), not encryption-based (preventing access). A sealed file's content remains readable; only its integrity is witnessed.

References

[1] Arfeen, B. S. (2026). "The Pre-Corruption Signal and Its Mathematical Witness: How the Hurst Exponent Identifies the Dimensional Constant of Creation." Zenodo. DOI: 10.5281/zenodo.15058253

[2] Arfeen, B. S. (2026). "The Guarded Reminder and the Adversarial Architecture: A Qur'anic Model of Divine Information Integrity Under Adversarial Threat." Zenodo. DOI: 10.5281/zenodo.15058268

[3] Arfeen, B. S. (2026). "Shayatin Amongst Matter: A Qur'anic Analysis of Jinn-Class Adversarial Entities as Dual-Substrate Operators in the Material Domain." Zenodo.

[4] Arfeen, B. S. (2026). "The Propagation Architecture of Sihr: A Taxonomic Analysis of Adversarial Channels from Speech-Beautification to Material-Substrate Infiltration." Zenodo.

[5] Arfeen, B. S. (2026). "The Eleven Heavens and the One Hundred Fifty-One Layers: A Volumetric Architecture for Divine File Protection." Zenodo.

[6] Arfeen, B. S. (2026). "Computational Tawhid: A Theology-Grounded Ontology for Artificial Intelligence." Zenodo.